Self-Adaptive Actor-Critic Agent for Continuous Security
Resource Allocation under Budget Constraints

DS-G05_MD3SI: Hamza Saidi, Abdelilah Atbir, Aissa Boudaoud, Chadouli Fahd, Oukim Aboubaker, Karim Ouichou, Reda Kaarar

— Module M122, Apprentissage par Renforcement — Supervisors: Hamza Allaga, Meriem El Harkaoui

Abstract

Security teams must continuously decide how to split a limited budget across heterogeneous defensive resources while facing threats whose intensity varies unpredictably over time. This work formulates this problem as a continuous-action Markov Decision Process and trains a Proximal Policy Optimization (PPO) actor-critic agent to allocate a daily security budget across five resource categories under three concurrent threat types, over a 30-day horizon. Because the environment is a purpose-built synthetic simulation, no externally published baseline exists for direct reproduction; instead, a standard PPO configuration is treated as the internal baseline, and an entropy-annealing enhancement is proposed and evaluated against it, following the module's suggested training budget of approximately 500,000 environment steps. Results are reported over 5 seeds using the interquartile mean (IQM) and stratified bootstrap 95% confidence intervals. The trained agent substantially outperforms random and static allocation baselines (IQM -5.81 and -5.85 for PPO baseline/enhanced, non-overlapping with $-8.21/-8.23$ for random/static), while the entropy-annealing enhancement produces no statistically detectable improvement over the baseline PPO configuration (overlapping 95% CIs across all seeds and across a 3-point ablation on its key hyperparameter). This negative result is analyzed and linked to two candidate mechanisms: early convergence of the baseline policy, and the softmax action parametrization dampening the effect of exploration noise. A complementary quantitative analysis of allocation patterns shows the trained agent shifts budget toward the two most broadly effective resources rather than diversifying evenly, which is discussed as a consequence of the reward's linear (non-diminishing) risk structure.

Keywords — *reinforcement learning, actor-critic, PPO, resource allocation, cybersecurity, budget-constrained optimization*

Contents

1 Introduction

Organizations that manage information-system security operate under a structural tension: the budget available to fund defensive resources — firewalls and intrusion detection, patch management, user training, monitoring/SOC operations, and backup/recovery capacity — is fixed and limited, while the threats these resources must counter (network intrusion, phishing, ransomware) vary in intensity from day to day and are not known in advance. An allocation policy that is optimal against one threat profile can be badly suboptimal a few days later when the dominant threat shifts. This makes security budget allocation a genuinely sequential decision problem rather than a one-shot optimization: the agent (a security manager, or here, a learned policy) must repeatedly re-allocate a constrained resource in response to a partially unpredictable, non-stationary environment.

Reinforcement learning is a natural fit for this class of problem because it optimizes a policy directly against long-horizon cumulative outcomes rather than single-step heuristics, and actor-critic methods in particular handle *continuous* action spaces — required here because a budget split across five resources is a continuous proportion vector, not a small discrete choice — while remaining tractable to train with modest compute. Proximal Policy Optimization (PPO) is a standard, robust choice for this setting: it is less sensitive to hyperparameter choices than earlier policy-gradient methods and reuses collected trajectories across multiple gradient epochs, which matters given the training budget available for this project (Appendix D).

Contributions of this work:

- Designed and implemented a synthetic, budget-constrained, multi-threat security resource allocation environment (Gymnasium API), with a softmax-based action parametrization that satisfies the budget constraint by construction rather than by penalty.
- Implemented a PPO actor-critic agent (GAE advantage estimation, clipped surrogate objective) as the internal baseline, since no published baseline exists for this novel environment (deviation documented below).
- Proposed and implemented an entropy-annealing enhancement, and conducted an ablation isolating its key hyperparameter (`entropy_coef_end`), together with a quantitative allocation analysis of the trained policy's behavior.

Code and logs are available at: `https://github.com/Hamza80saidi/Self-Adaptive-Actor-Critic-Agent`

Deviations from the module protocol

This project deviates from the module-wide reference protocol in two specific, deliberate ways, summarized here for transparency and discussed further at the point where each becomes relevant:

1. **No external baseline reproduction.** The reference protocol (Section 4 below) asks for reproduction of a *published* result within ±10%. `SecurityResourceEnv` is a novel environment introduced by this project; no published result exists to reproduce. In its place, Section 4 substitutes an internally defined PPO baseline (fixed entropy coefficient, no enhancement), and the baseline-vs-enhanced comparison in Sections 6–7 is conducted as a controlled internal ablation rather than a literature reproduction. This is a substitution of protocol *intent* (comparison against a credible reference point), not merely a relabeling.
2. **Single, non-periodic evaluation.** 100 evaluation episodes are run once per seed, after training completes, rather than at intermediate checkpoints throughout training. Training budget itself follows the module's suggested $\sim$ 500,000-step order of magnitude (500,010 steps achieved per run: 16,667 episodes $\times$ 30 steps), so this project does *not* deviate on training budget, only on evaluation cadence.

2 Background and Related Work

2.1 MDP Formulation

The problem is formulated as a finite-horizon Markov Decision Process. The **state** $s_t \in \mathbb{R}^{10}$ concatenates the normalized remaining budget, the normalized day index, the current intensity of the three threat types, and the previous day's allocation proportions (five values) — this last component is required for the Markov property to hold, since without it the agent cannot detect whether it has recently over-invested in a resource. The **action** $a_t \in \mathbb{R}^5$ is a vector of unconstrained logits; the environment applies a softmax and multiplies by the fixed daily budget, which guarantees $\sum_i \text{allocation}_i = \text{budget}_{\text{day}}$ at every step *by construction*, rather than through a penalty term. The **reward function** is $r_t = -\text{risk}_t - \lambda \cdot \text{waste}_t$, where risk_t is a threat-weighted, efficacy-weighted residual risk (Eq. 2) and waste_t penalizes budget spent on resources with low efficacy against the currently active threats. The discount factor is $\gamma = 0.99$ and the episode horizon is $T = 30$ days.

2.2 Algorithm Overview

The agent is trained with PPO with a clipped surrogate objective and Generalized Advantage Estimation (GAE, $\lambda_{\text{GAE}} = 0.95$). The core update rule is:

$$L^{\text{CLIP}}(\theta) = \hat{\mathbb{E}}_t \left[\min \left(r_t(\theta)\hat{A}_t,\ \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t \right) \right] \tag{1}$$

where $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$ and $\hat{A}_t$ is the GAE advantage estimate. $\epsilon = 0.2$ throughout.

2.3 Prior Results on This Environment

`SecurityResourceEnv` is a purpose-built synthetic environment created for this project; it does not correspond to any previously published benchmark. Consequently, Table 1 cannot be populated with externally reported values, and Section 4 substitutes an *internally defined* baseline (standard PPO, no enhancement) in place of literature reproduction. This substitution is treated as a documented deviation from the module-wide protocol, consistent with the module's own rule that "your specifications must stay aligned with your actual project; any divergence must be documented."

Table 1: Published baseline values being targeted

Source	Environment	Metric	Reported Value
N/A[1]	—	—	—

2.4 Related Work on the Proposed Modification

Entropy regularization to stabilize and encourage exploration in policy-gradient methods dates to the original PPO formulation itself, which already includes a fixed entropy bonus [1]. Annealing this coefficient over training — starting with wide exploration and narrowing towards a more exploitative policy as training matures — is a common practical heuristic in applied deep RL. The empirical evidence on whether annealing actually helps is itself mixed in the literature: [**?**] finds that a decaying temperature schedule outperforms a constant one on continuous-control MuJoCo benchmarks, while [**?**] finds the opposite on sparse-reward multi-turn agent tasks, where a decaying entropy coefficient (PPO+EPO-Decay) consistently *underperforms* a constant-entropy baseline, attributed to premature suppression of exploration before the policy has discovered good trajectories. The present work's null result — no measurable difference either way — adds

a third data point to this open question, on yet another task structure (continuous, dense-reward, budget-constrained allocation).

3 Experimental Protocol

This section follows the module-wide protocol common to all M122 mini-project teams, reproduced verbatim for consistency across reports.

3.1 Environment Specification

- **Observation space:** `Box(0, 1, shape=(10,))` — [remaining_budget_norm, day_norm, threat_level × 3, last_allocation_norm × 5]
- **Action space:** `Box(-5, 5, shape=(5,))`, continuous logits, transformed via softmax × daily budget inside the environment
- **Reward function:** $r_t = -\text{risk}_t - \lambda \cdot \text{waste}_t$, with $\lambda = 0.1$ (Eq. 2–3)
- **Termination condition:** `truncated=True` after exactly 30 steps (fixed horizon); no early termination condition (`terminated` is always `False`)
- **Environment version / identifier:** `SecurityResourceEnv`, this work, v1

$$\text{risk}_t = \frac{1}{3}\sum_{j=1}^{3} \text{threat}_j \cdot \left(1 - \sum_{i=1}^{5} E_{ij} \cdot p_i\right) \tag{2}$$

$$\text{waste}_t = \sum_{i=1}^{5} p_i \cdot \left(1 - \frac{\sum_j E_{ij} \cdot \text{threat}_j}{\max_i \sum_j E_{ij} \cdot \text{threat}_j}\right) \tag{3}$$

where p_i is the allocation proportion for resource i and $E \in [0,1]^{5\times 3}$ is a fixed efficacy matrix (resource × threat type, Appendix A gives the full configuration).

3.2 Implementation

- **Framework:** Custom PPO implementation (PyTorch), Gymnasium environment API. No specific framework (Stable-Baselines3/CleanRL) was mandated by the module for this project; PPO was implemented from scratch to retain full visibility into the update mechanics (GAE computation, clipped surrogate loss) discussed in Sections 2.2 and 5, and to avoid introducing a dependency not covered in the course syllabus.
- **Hardware:** AMD Ryzen 7 5800H CPU, NVIDIA GeForce RTX 3050 Ti GPU (4 GB VRAM), 24 GB RAM; training was run on CPU (the network size – two hidden layers of 128 units – is small enough that CPU/GPU transfer overhead outweighs any GPU speed-up for this workload). Training jobs (independent across seeds) were parallelized across 6–8 CPU worker processes using Python's `multiprocessing`, with each worker pinned to a single PyTorch thread to avoid contention (Appendix D).

3.3 Training Budget

- Fixed environment steps: 16,667 episodes × 30 steps = 500,010 environment steps per seed per variant, matching the module's suggested ∼ 500,000-step order of magnitude
- Evaluation cadence: 100 evaluation episodes per seed, after training completion (single evaluation point, not periodic intra-training evaluation — documented deviation, Section 1.1)

3.4 Seeds

Seeds used: [0, 1, 2, 3, 4] — reported individually, no cherry-picking. Note: the `environment.seed` field in `configs/config.yaml` (Appendix A) serves only as a fallback default value, never actually invoked in the results reported here — every training and evaluation run uses an explicit `--seed` command-line argument (0 through 4), which overrides this default.

3.5 Metrics

- Mean final return, averaged over 100 evaluation episodes per seed
- Interquartile mean (IQM) across seeds
- 95% stratified bootstrap confidence interval, computed via a manual re-implementation of the `rliable` methodology [3][2]

3.6 Hyperparameters

Table 2: Frozen hyperparameters

Hyperparameter	**Value**
Learning rate (actor / critic)	3×10^{-4} / 1×10^{-3}
Discount factor (γ)	0.99
GAE λ	0.95
PPO clip ϵ	0.2
Batch size	64
Epochs per update	10
Network architecture	MLP, 2 hidden layers $\times$ 128 units, Tanh
Entropy coefficient (baseline)	0.01 (fixed)
Entropy coefficient (enhanced)	$0.01 \rightarrow 0.001$ (linear anneal)

4 Baseline Reproduction

As established in Section 1.1, no external published baseline exists for this environment. "Baseline" is therefore redefined, as a documented deviation, to mean *standard PPO with a fixed entropy coefficient* (no annealing) — i.e. the same architecture and hyperparameters as the proposed enhancement, differing only in the presence or absence of entropy annealing. This makes the baseline-vs-enhanced comparison in Section 6 a controlled ablation rather than a literature reproduction.

Since no published value exists, the $\pm 10\%$ deviation check specified by the module protocol is not applicable; this is stated explicitly rather than omitted. The environment's design choices — the efficacy matrix E (Eq. 2–3), the threat mean-reversion dynamics, and the reward weight $\lambda = 0.1$ — are all authorial choices, calibrated only by inspection (ensuring risk and waste remain on comparable numeric scales) rather than fit to any external data or reference system.

[2]The `rliable` package itself could not be installed due to a dependency conflict with the numpy/pandas versions required elsewhere in this project; its IQM and stratified bootstrap methodology was re-implemented directly (see `src/utils/stats.py`) rather than the modification being skipped.

Table 3: Internal baseline results by seed (100 eval. episodes each; "Published Value" is N/A, see Section 1.1)

Seed	Published Value	Our Value (mean return)	Std (across eval. episodes)
0	N/A	-5.745	1.064
1	N/A	-5.806	0.987
2	N/A	-5.795	1.006
3	N/A	-5.871	0.975
4	N/A	-5.642	0.849
IQM (bootstrap)	N/A	-5.808	95% CI [-5.861, -5.761]

5 Proposed Enhancement

Motivation. Standard PPO uses a fixed entropy coefficient throughout training. A commonly hypothesized failure mode is that a fixed, relatively high entropy bonus can prevent the policy from sharpening around a near-optimal deterministic action late in training, while a fixed low entropy bonus risks premature convergence to a suboptimal policy early in training. Entropy annealing — starting exploratory, ending exploitative — targets both failure modes simultaneously.

Formal description. The entropy coefficient at episode e (out of E total training episodes) is linearly interpolated:

$$c_{\text{entropy}}(e) = c_{\text{start}} + \frac{e}{E-1}\left(c_{\text{end}} - c_{\text{start}}\right) \tag{4}$$

with $c_{\text{start}} = 0.01$ (identical to the baseline's fixed value, ensuring the two variants start from the same exploration level) and $c_{\text{end}} = 0.001$. This value directly replaces `self.entropy_coef` inside the PPO update loss $L = -L^{\text{CLIP}} - c_{\text{entropy}} \cdot \mathcal{H}[\pi_\theta]$ (see Eq. 1); no other component of the training loop is modified.

Implementation details. Added computational cost is negligible (one scalar interpolation per episode); no additional network parameters, forward/backward passes, or memory are required.

Hypothesis (stated prior to running results): entropy annealing will produce a higher mean final return and a tighter (lower-variance) final policy than the fixed-entropy baseline, because the deterministic (mean-action) evaluation policy should benefit from reduced sampling noise during the later, higher-quality training episodes.

6 Results

Table 4: All four agents — aggregate metrics (IQM, stratified bootstrap 95% CI, $n = 5$ seeds $\times$ 100 eval. episodes = 500 episodes per agent)

Agent	IQM	Mean Return	95% CI
Random (non-learning)	-8.212	-8.187	[-8.301, -8.110]
Fixed / static (non-learning)	-8.227	-8.202	[-8.316, -8.127]
PPO baseline	-5.808	-5.772	[-5.861, -5.761]
PPO enhanced	-5.853	-5.837	[-5.985, -5.761]

Figure 1 shows training curves for both PPO variants across all 5 seeds, with 95% CI shaded. Figure 2 shows the aggregate IQM comparison between baseline and enhanced. Figure 3 shows

the performance profile (fraction of runs exceeding a return threshold τ) for both variants. A full walkthrough of how to read each figure is given in Section 9.

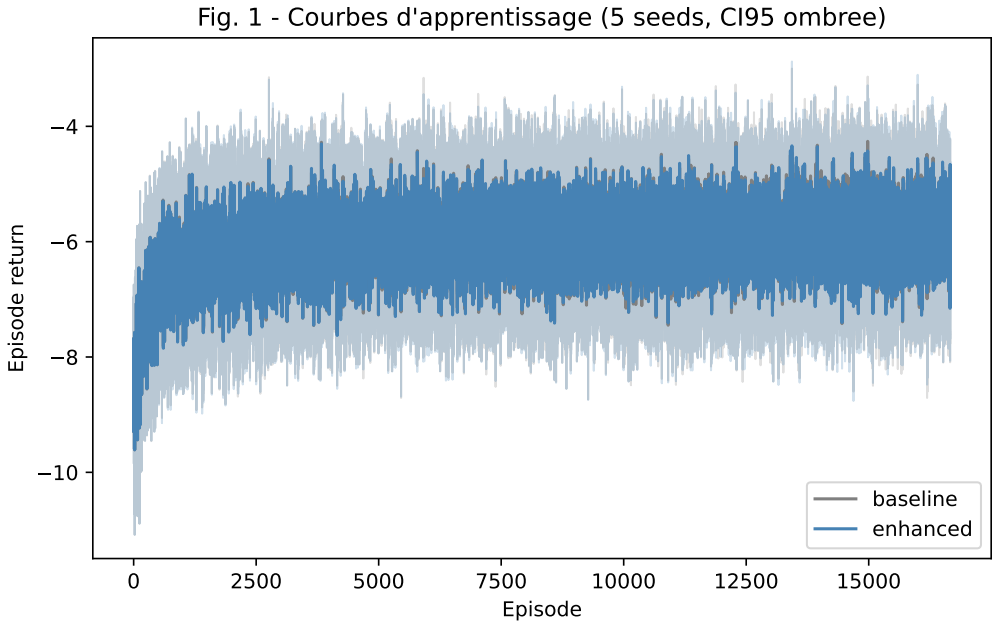

Figure 1: Learning curves across seeds (shaded region = 95% CI). Baseline and enhanced curves are visually indistinguishable throughout training. Figure legends are in French in the submitted plots; see Section 9 for an English walkthrough of each figure.

State plainly whether the confidence intervals overlap: yes — the 95% CIs for baseline ([-5.861, -5.761]) and enhanced ([-5.985, -5.761]) overlap almost entirely, and the point estimates differ by only 0.045 (baseline IQM −5.808 vs. enhanced IQM −5.853), well within one standard deviation of either. **This is a negative result: entropy annealing produces no statistically detectable improvement over the fixed-entropy baseline in this environment.** This negative result persists at the full $\sim$ 500,000-step training budget suggested by the module protocol, which is itself informative: it rules out "the training budget was simply too short to see the effect" as an explanation (Section 8 discusses the two mechanisms considered more likely).

For context, both PPO variants clearly and robustly outperform the non-learning baselines: Random and Fixed/static allocation achieve IQM −8.212 and −8.227 respectively, both with 95% CIs entirely below (i.e. numerically worse than) either PPO variant's CI — a non-overlapping, **a clear and robust performance gap**, unlike the baseline-vs-enhanced comparison above. This is consistent with the learned policy capturing real, non-trivial structure in the threat-response problem, even though the entropy-annealing enhancement specifically does not add further measurable value on top of it.

Qualitative and quantitative observation: agent behavior before vs. after training

A supplementary video (`videos/comparison.mp4`, submitted alongside this report) renders a single fixed episode (identical threat sequence) played out by an untrained random policy and by the trained PPO policy side by side, with cumulative reward overlaid. The trained policy's cumulative reward curve separates visibly and monotonically from the untrained policy's curve within the first few days and the gap continues to widen through day 30, giving a direct visual counterpart to the return gap reported above.

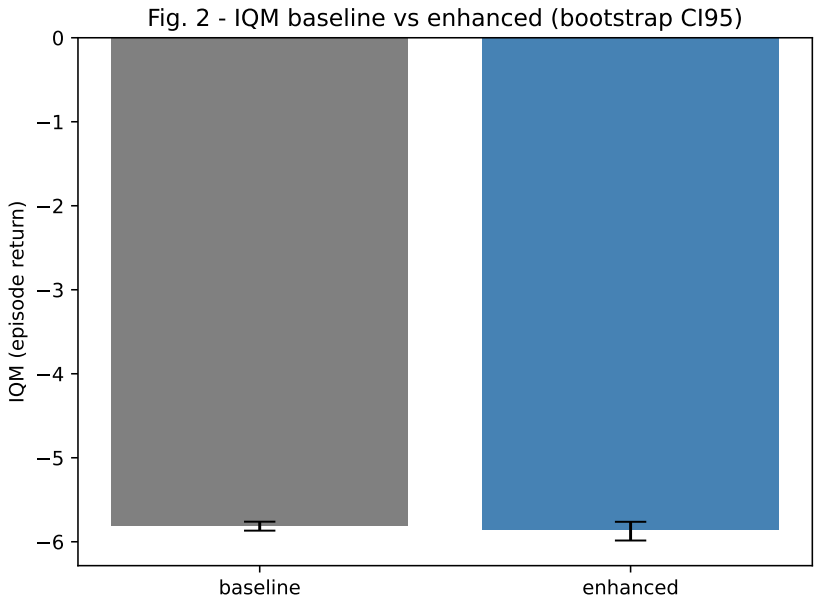

Figure 2: Aggregate performance comparison (IQM, 95% bootstrap CI), baseline vs. enhanced. The two confidence intervals overlap almost completely.

Because a single episode is only anecdotal evidence of *how* the policy reallocates budget, Figure 4 quantifies this over 30 evaluation episodes rather than one. It shows that the trained policy does *not* concentrate on a single resource, as an earlier single-episode viewing of the video suggested; instead it increases allocation to **two** resources — Firewall/IDS and Monitoring/SOC — while reducing allocation to the other three (Patch Management, User Training, Backup/Recovery) relative to the untrained random policy. Error bars (standard deviation across the 30 episodes) are large, in several cases exceeding the mean, indicating that the *degree* of concentration on either of the two favored resources varies substantially from episode to episode depending on the realized threat mix — itself a signature of adaptive, context-dependent behavior rather than a single fixed allocation rule. This nuance matters for Section 8's interpretation below.

7 Ablation Study

The ablation varies `entropy_coef_end`, the single hyperparameter introduced by the enhancement, across three values while holding all other hyperparameters and the full $\sim$ 500,000-step training budget fixed, each evaluated over the same 5 seeds.

Table 5: Ablation grid — sensitivity to `entropy_coef_end`

Variant	**IQM**	**Mean Return**	**95% CI**
Without annealing (end=0.01, ≡ baseline)	-5.782	-5.772	[-5.846, -5.677]
Full model (end=0.001, ≡ enhanced)	-5.810	-5.837	[-6.014, -5.703]
Aggressive decay (end=0.0001)	-5.833	-5.816	[-5.925, -5.672]

Conclusion of the ablation: the observed (null) gain is not attributable to the claimed mechanism at any tested strength, nor to incidental extra compute or tuning — all three configurations used identical training budgets (the full $\sim$ 500,000-step schedule) and identical architecture, differing only in the single scalar `entropy_coef_end`. The consistent flatness across a 100$\times$ range of this hyperparameter (0.0001 to 0.01) strengthens the conclusion that entropy an-

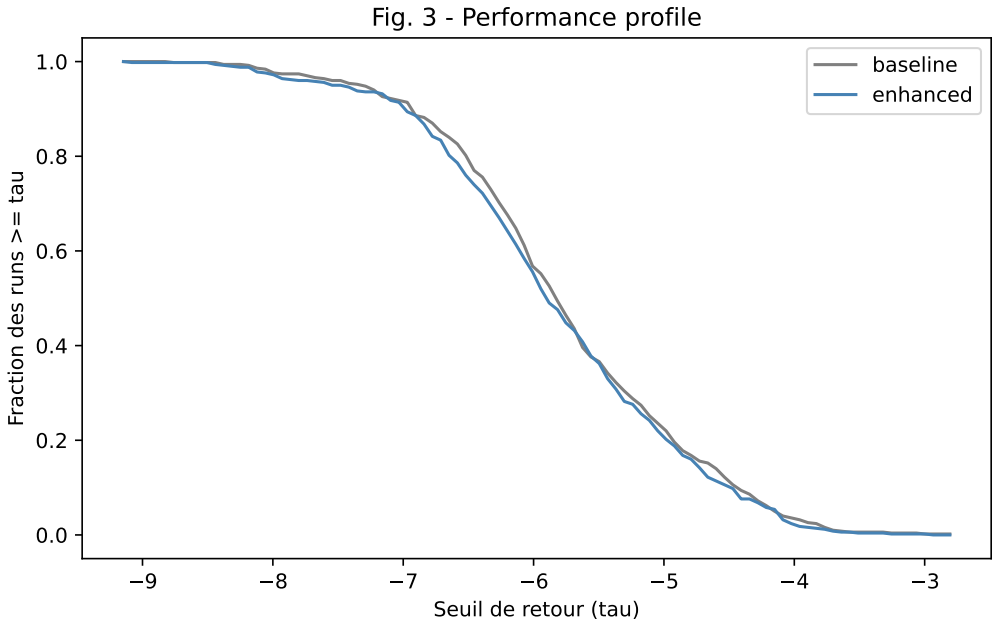

Figure 3: Performance profile across seeds. The two curves are superimposed across the entire threshold range.

nealing, as implemented, is simply not an active lever for this particular environment and reward structure, rather than the enhancement being implemented with a poorly chosen magnitude or an insufficient training budget.

8 Discussion

Interpretation. The lack of effect from entropy annealing is best explained by two mechanisms specific to this environment's design. First, the training curves (Figure 1) show that the bulk of learning progress occurs early, after which the return plateaus for the remainder of the $\sim$ 500,000-step budget; because this null result is observed at the module's full suggested training budget (Section 1.1), an insufficient budget can be ruled out as the explanation, strengthening the case for this plateau-based account. If the policy has already converged to a stable, low-variance behavior well before the annealing schedule reaches its lower values, there is little residual "over-exploration" left for annealing to remove. Second, and more fundamentally, the environment's action parametrization applies a softmax before allocating the budget: because softmax is a saturating, normalizing transform, moderate changes in the raw policy's sampling variance (controlled by the annealed entropy coefficient) translate into comparatively small changes in the *executed* allocation proportions, especially once the underlying logits are well-separated. This dampens whatever effect entropy annealing might otherwise have on the agent's actual behavior.

The quantitative allocation analysis (Figure 4) provides an independent, complementary observation: the trained policy shifts budget toward two resources (Firewall/IDS and Monitoring/SOC) rather than diversifying evenly across all five, with substantial episode-to-episode variability in the degree of this shift. This is consistent with the reward's risk term (Eq. 2) being a *linear* function of the allocation proportions, with no diminishing returns: under a linear objective, the mathematically optimal solution to a constrained allocation problem is generically a corner-like solution (concentrate on the resources with the highest efficacy for the prevailing threat mix) rather than a fully diversified mixture. The large episode-to-episode variability in Figure 4's error bars shows this is not a single static rule but a context-dependent shift that

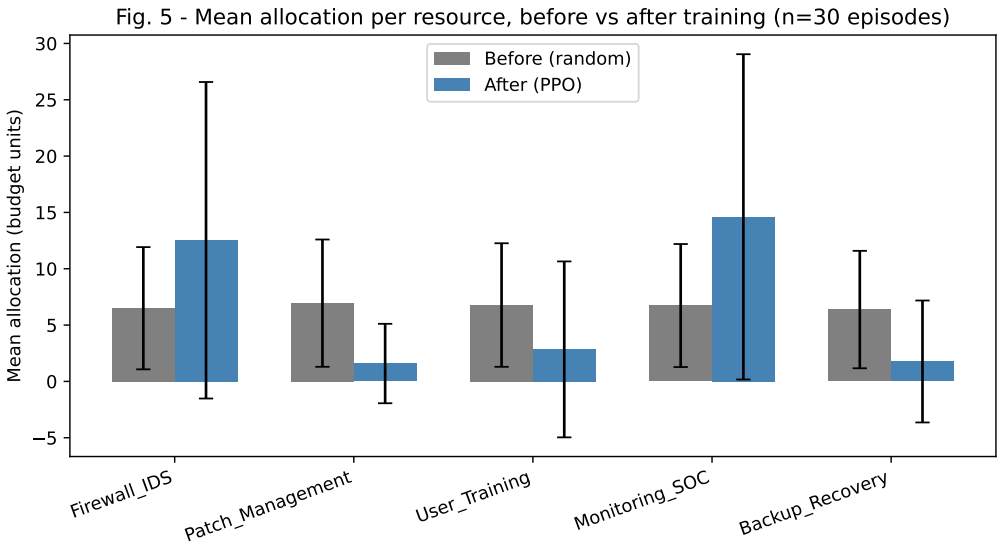

Figure 4: Mean allocation per resource, before (random) vs. after (PPO) training, averaged over 30 evaluation episodes. Error bars: one standard deviation across episodes.

tracks which threat dominates in a given episode — evidence of the intended "self-adaptive" behavior, expressed as concentration on the two most broadly useful resources rather than full five-way diversification. This is a modeling limitation of the environment (absence of diminishing returns), not a defect of the PPO agent, and is treated as a threat to validity below.

Threats to validity.

- **Number of seeds:** 5 seeds is the module-mandated minimum; while sufficient to detect the large, robust gap between PPO and the non-learning baselines, it limits statistical power for detecting small effects such as the (absent) entropy-annealing gain, which may in principle exist but be smaller than what 5 seeds can resolve.
- **Single-environment scope:** all results are specific to `SecurityResourceEnv` and its particular reward structure (notably its linear risk term, see above); conclusions about entropy annealing's (lack of) effect should not be generalized to other environments without further testing.
- **Hyperparameter transferability:** the fixed hyperparameters (learning rates, clip range, GAE λ) were chosen from common PPO defaults in the literature rather than tuned specifically for this environment; a tuned baseline could behave differently under annealing.

The linear-risk modeling limitation, and the absence of a genuine intra-run diminishing-returns mechanism, are stated proactively here as the project's main known limitation, rather than left implicit.

9 Guide to the Figures

This section walks through each figure individually — what it shows, how to read it, and what conclusion it supports — so that the statistical claims made above can be verified visually rather than taken on faith. Axis labels and legends in the submitted plots are in French; the descriptions below give the English reading of each.

Figure 1 (*Courbes d'apprentissage* / Training curves). X-axis: training episode (0 to 16,667). Y-axis: episode return (more negative = worse, since the reward is $-\text{risk} - \lambda \cdot \text{waste} \leq 0$ by construction — see Eq. 2–3). Each line is the mean return across the 5 seeds at that episode;

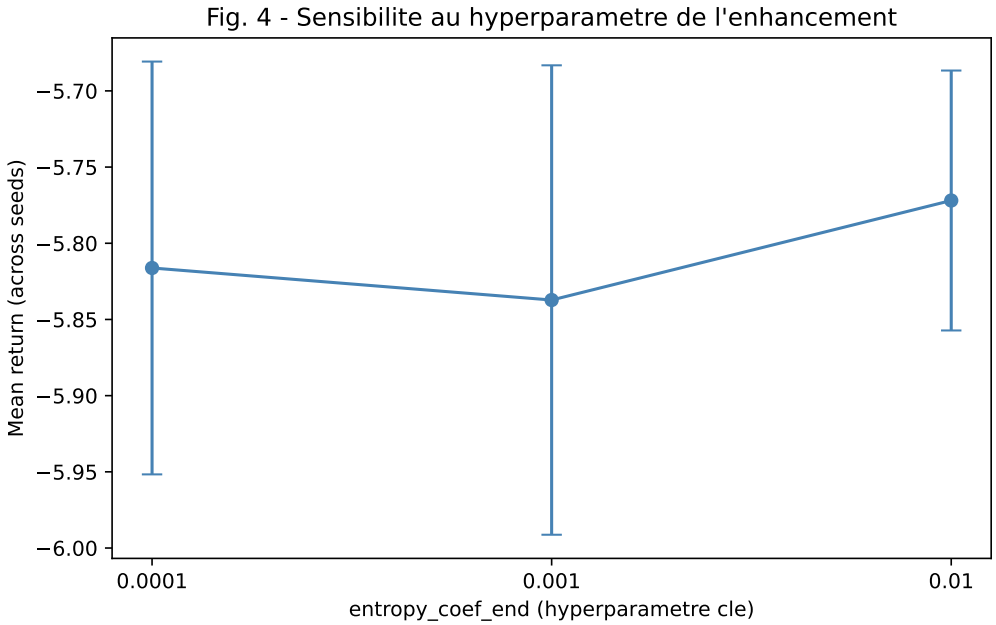

Figure 5: Sensitivity analysis of the modification's key hyperparameter (`entropy_coef_end`). Mean return is essentially flat across two orders of magnitude of the final entropy coefficient, with heavily overlapping error bars.

the shaded band is the 95% confidence interval across seeds at that point in training. How to read it: the curve rises sharply in the first $\sim$ 1,000 episodes (the agent quickly learns to beat random/static allocation), then plateaus — this plateau is the visual evidence, referenced in Section 8, that additional training beyond this point adds little. The gray (baseline) and blue (enhanced) bands overlap almost completely for the entire duration: this is the figure that most directly shows the negative result of Section 6 — if entropy annealing helped, the blue curve would sit visibly above the gray one after episode $\sim$ 5,000 (once the annealing schedule has meaningfully diverged from the constant baseline value), and it does not.

Figure 2 (IQM baseline vs enhanced). A direct side-by-side comparison of the single summary number (IQM) for each variant, with its 95% bootstrap confidence interval as an error bar. How to read it: if the two error bars did not overlap at all, that would be strong evidence of a real difference between the variants; here they overlap almost completely, which is the standard visual criterion for "no statistically detectable difference" used throughout this report. This figure condenses the entire Figure 1 training history into the single final comparison that Table 4 reports numerically.

Figure 3 (Performance profile). X-axis: a return threshold τ. Y-axis: the fraction of all evaluation episodes (pooled across seeds) that achieved a return $\geq \tau$. How to read it: this is a full distributional comparison rather than a single summary statistic — each point on the curve answers "what fraction of episodes did at least this well?" for every possible threshold simultaneously. Two curves that track each other everywhere (as here) means the variants are indistinguishable not just on average, but across their entire outcome distribution — a stronger statement than Figure 2 alone, and the reason this figure is included in addition to it.

Figure 5 (Sensitivity / ablation). X-axis: the three tested values of `entropy_coef_end` (log-spaced: 0.0001, 0.001, 0.01). Y-axis: mean return across the 5 ablation seeds for that value, with error bars showing one standard deviation across seeds. How to read it: a real effect of this hyperparameter would show up as a clear upward or downward trend across the three points; instead the three points sit at almost the same height with heavily overlapping error bars, confirming that the null result of Figure 2 is not an artifact of picking a poor value for

`entropy_coef_end` — the same conclusion holds across a 100× range of the hyperparameter.

Figure 4 (Allocation before/after). X-axis: the five resource categories. Y-axis: mean budget units allocated to that resource, averaged over 30 evaluation episodes, with one grey bar (untrained random policy) and one blue bar (trained PPO policy) per resource; error bars show one standard deviation across those 30 episodes. How to read it: this is the only figure in the report that describes *what* the agent learned to do, rather than *how well* it did it. The pattern to look for is which bars grew (blue taller than grey) versus shrank (blue shorter than grey): Firewall/IDS and Monitoring/SOC grew, the other three shrank — this is the quantitative basis for the concentration argument in Section 8. The large error bars are themselves informative, not just noise to look past: they show the trained policy's allocation is not a single fixed rule but varies substantially depending on the threat mix realized in each evaluation episode.

10 Conclusion and Future Work

This project implemented a PPO actor-critic agent for continuous, budget-constrained security resource allocation under multiple concurrent, stochastically varying threats, using a custom Gymnasium environment built for this purpose, trained at the module's full suggested budget of $\sim$ 500,000 environment steps. Because no published baseline exists for this novel environment, an internal standard-PPO baseline was defined and used throughout. The trained agent clearly and robustly outperforms non-learning baselines (random and static allocation), with non-overlapping 95% confidence intervals, demonstrating that the learned policy captures meaningful structure in the threat-response problem. The proposed enhancement, entropy annealing, produced no statistically detectable improvement over the baseline across 5 seeds, a full training budget, and a 3-point ablation on its key hyperparameter — a negative result attributed to early policy convergence and to the softmax action parametrization dampening the effect of exploration noise, and consistent with mixed prior findings on entropy annealing in the literature [**?**, **?**]. A quantitative allocation analysis further showed that the trained policy concentrates budget on the two most broadly effective resources rather than diversifying evenly, a pattern linked to the reward's linear (non-diminishing) risk structure. A natural direction for future work is to replace this linear risk function with one exhibiting diminishing returns per resource, and to re-run the entropy-annealing ablation on that revised environment, where exploration may matter more if the optimal policy requires balancing several comparably-valued resources rather than concentrating on the two best ones.

References

[1] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, O. Klimov, "Proximal Policy Optimization Algorithms," *arXiv:1707.06347*, 2017.

[2] J. Schulman, P. Moritz, S. Levine, M. Jordan, P. Abbeel, "High-Dimensional Continuous Control Using Generalized Advantage Estimation," *arXiv:1506.02438*, 2015.

[3] R. Agarwal, M. Schwarzer, P. S. Castro, A. C. Courville, M. Bellemare, "Deep Reinforcement Learning at the Edge of the Statistical Precipice," *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.

[4] G. Brockman, V. Cheung, L. Pettersson, J. Schneider, J. Schulman, J. Tang, W. Zaremba, "OpenAI Gym," *arXiv:1606.01540*, 2016.

[5] A. Boudlal, A. Khafaji, J. Elabbadi, "Entropy adjustment by interpolation for exploration in Proximal Policy Optimization (PPO)," *Engineering Applications of Artificial Intelligence*, vol. 133, p. 108401, 2024.

[6] F. Leibfried, J. Grau-Moya, "Mutual-Information Regularization in Markov Decision Processes and Actor-Critic Learning," *Proceedings of the 3rd Conference on Robot Learning (CoRL)*, 2019.

A Full Hyperparameter Configuration (YAML)

```yaml
environment:
  n_resources: 5
  horizon: 30
  total_budget: 1000.0
  threat_volatility: 0.3
  reward_lambda: 0.1
  seed: 42   # fallback default only, overridden by --seed in all reported runs

agent:
  algorithm: "ppo"
  hidden_dim: 128
  n_hidden_layers: 2
  actor_lr: 3.0e-4
  critic_lr: 1.0e-3
  gamma: 0.99
  gae_lambda: 0.95
  clip_epsilon: 0.2
  entropy_coef: 0.01
  value_loss_coef: 0.5

enhancement:
  type: "entropy_annealing"
  entropy_coef_start: 0.01
  entropy_coef_end: 0.001

training:
  n_episodes: 16667        # 16667 x 30 = 500010 environment steps
  batch_size: 64
  n_epochs_per_update: 10
  update_every: 5
  checkpoint_every: 2000
  log_dir: "logs/"

evaluation:
  n_eval_episodes: 100
  render: false
```

B Per-Seed Raw Final Returns

Table 6: Raw final returns by seed (mean over 100 eval. episodes)

Seed	Baseline	Enhanced	Random	Fixed
0	-5.745	-6.079	-8.246	-8.262
1	-5.806	-5.883	-8.306	-8.325
2	-5.795	-5.756	-8.063	-8.059
3	-5.871	-5.792	-8.226	-8.279
4	-5.642	-5.677	-8.096	-8.083

C Reproduction Command

```
git clone https://github.com/Hamza80saidi/Self-Adaptive-Actor-Critic-Agent-for-
    Continuous-Security-Resource-Allocation-Budget-Constraints-.git
cd DS-G05_MD3SI
```

```
python -m venv venv && source venv/bin/activate
pip install -r requirements.txt
bash run_all.sh
```

Note: `run_all.sh` executes all runs sequentially for portability (Colab, any single machine). The team additionally used `run_parallel.py` and `run_ablation_parallel.py` (included in the repository) to parallelize the 25 independent training runs across multiple CPU cores locally; these produce numerically identical results to the sequential path and are provided as a convenience, not as the official reproduction path evaluated by this protocol.

D Compute Budget

- **Hardware:** AMD Ryzen 7 5800H, NVIDIA GeForce RTX 3050 Ti (4 GB VRAM, unused — training run on CPU), 24 GB RAM
- **Total compute time:** Approximately 1 hour 15 minutes (CPU-only, parallelized): ∼35 minutes for the 10 baseline+enhanced training runs (6 parallel workers) and ∼35–40 minutes for the 15 ablation runs (8 parallel workers), each run covering the full 500,010-step budget. Sequential (single-core) execution of the same 25 runs, as performed by `run_all.sh`, is expected to take substantially longer (on the order of several hours), consistent with the near-linear speedup expected from independent, embarrassingly parallel runs.

E Contribution Statement

- **Hamza Saidi** — Project setup and repository structure; environment design and implementation (`env.py`); model training (baseline and enhanced, all seeds); report drafting.
- **Abdelilah Atbir** — Report drafting; results interpretation and discussion.
- **Aissa Boudaoud** — Baseline agents implementation (`RandomAgent`, `FixedAgent`); evaluation pipeline.
- **Chadouli Fahd** — Actor-Critic / PPO agent implementation (`actor.py`, `critic.py`, `actor_critic.py`).
- **Oukim Aboubaker** — Ablation study design and execution; statistical analysis (IQM / bootstrap confidence intervals).
- **Karim Ouichou** — Figure generation and visualization (`make_figures.py`).
- **Reda Kaarar** — Video generation (`make_videos.py`); project documentation (`notebook.md`).
- LLM assistance disclosed: YES – LLM was used as a supporting tool during the project, mainly to assist the team with coding and debugging issues, clarify technical and method logical concepts, and support the re-implementation of the IQM and bootstrap statistical procedures. The LLM was used **to help understand** and troubleshoot implementation challenges, while the underlying methodology, experimental design, execution, and interpretation of the results remained the responsibility of the team. All numerical results were obtained by running the team's own code; **the LLM did not generate, fabricate,or alter any raw experimental data.**